

Chapter: 3 JavaScript

JavaScript:

Introduction to JavaScript:

JavaScript was invented by **Brendan Eich** in 1995.

JavaScript Tutorial for beginners and professionals is a solution of client side dynamic pages.

JavaScript is *an object-based scripting language* that is lightweight and cross-platform.

JavaScript is not compiled but translated. The JavaScript Translator (embedded in browser) is responsible to translate the JavaScript code.

Where JavaScript is used

JavaScript is used to create interactive websites. It is mainly used for:

- Client-side validation
- Dynamic drop-down menus
- Displaying data and time
- Displaying popup windows and dialog boxes (like alert dialog box, confirm dialog box and prompt dialog box)
- Displaying clocks etc.

JavaScript Example

1. `<h2>Welcome to JavaScript</h2>`
2. `<script>`
3. `document.write("Hello JavaScript by JavaScript");`
4. `</script>`

JavaScript Variable:

1. [JavaScript variable](#)
2. [JavaScript Local variable](#)
3. [JavaScript Global variable](#)

A **JavaScript variable** is simply a name of storage location. There are two types of variables in JavaScript : local variable and global variable.

There are some rules while declaring a JavaScript variable (also known as identifiers).

1. Name must start with a letter (a to z or A to Z), underscore(_), or dollar(\$) sign.
2. After first letter we can use digits (0 to 9), for example value1.
3. JavaScript variables are case sensitive, for example x and X are different variables.

Correct JavaScript variables

1. `var x = 10;`
2. `var _value="sonoo";`

Incorrect JavaScript variables

1. `var 123=30;`
2. `var *aa=320;`

Example of JavaScript variable

Let's see a simple example of JavaScript variable.

1. `<script>`
2. `var x = 10;`
3. `var y = 20;`

4. `var z=x+y;`
5. `document.write(z);`
6. `</script>`

Output of the above example

30

JavaScript local variable

A JavaScript local variable is declared inside block or function. It is accessible within the function or block only. For example:

1. `<script>`
2. `function abc(){`
3. `var x=10;//local variable`
4. `}`
5. `</script>`

Or,

1. `<script>`
2. `If(10<13){`
3. `var y=20;//JavaScript local variable`
4. `}`
5. `</script>`

JavaScript global variable

A **JavaScript global variable** is accessible from any function. A variable i.e. declared outside the function or declared with window object is known as global variable. For example:

1. `<script>`
2. `var data=200;//global variable`
3. `function a(){`
4. `document.writeln(data);`
5. `}`
6. `function b(){`
7. `document.writeln(data);`
8. `}`
9. `a();//calling JavaScript function`
10. `b();`
11. `</script>`

o/p: 200 200

Javascript Data Types:

JavaScript provides different **data types** to hold different types of values. There are two types of data types in JavaScript.

1. Primitive data type
2. Non-primitive (reference) data type

JavaScript is a **dynamic type language**, means you don't need to specify type of the variable because it is dynamically used by JavaScript engine. You need to use **var** here to specify the data type. It can hold any type of values such as numbers, strings etc. For example:

1. `var a=40;//holding number`
2. `var b="Rahul";//holding string`

JavaScript primitive data types

There are five types of primitive data types in JavaScript. They are as follows:

Data Type	Description
String	represents sequence of characters e.g. "hello"
Number	represents numeric values e.g. 100
Boolean	represents boolean value either false or true
Undefined	represents undefined value
Null	represents null i.e. no value at all

JavaScript non-primitive data types

The non-primitive data types are as follows:

Data Type	Description
Object	represents instance through which we can access members
Array	represents group of similar values
RegExp	represents regular expression

JavaScript Operators:

JavaScript operators are symbols that are used to perform operations on operands. For example:

1. `var sum=10+20;`

Here, + is the arithmetic operator and = is the assignment operator.

There are following types of operators in JavaScript.

1. Arithmetic Operators
2. Comparison (Relational) Operators
3. Bitwise Operators
4. Logical Operators
5. Assignment Operators
6. Special Operators

JavaScript Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations on the operands. The following operators are known as JavaScript arithmetic operators.

Operator	Description	Example
+	Addition	<code>10+20 = 30</code>
-	Subtraction	<code>20-10 = 10</code>
*	Multiplication	<code>10*20 = 200</code>
/	Division	<code>20/10 = 2</code>
%	Modulus (Remainder)	<code>20%10 = 0</code>
++	Increment	<code>var a=10; a++; Now a = 11</code>
--	Decrement	<code>var a=10; a--; Now a = 9</code>

JavaScript Comparison Operators

The JavaScript comparison operator compares the two operands. The comparison operators are as follows:

Operator	Description	Example
----------	-------------	---------

==	Is equal to	10==20 = false
===	Identical (equal and of same type)	10===20 = false
!=	Not equal to	10!=20 = true
!==	Not Identical	20!==20 = false
>	Greater than	20>10 = true
>=	Greater than or equal to	20>=10 = true
<	Less than	20<10 = false
<=	Less than or equal to	20<=10 = false

JavaScript Bitwise Operators

The bitwise operators perform bitwise operations on operands. The bitwise operators are as follows:

Operator	Description	Example
&	Bitwise AND	(10==20 & 20==33) = false
	Bitwise OR	(10==20 20==33) = false
^	Bitwise XOR	(10==20 ^ 20==33) = false
~	Bitwise NOT	(~10) = -10
<<	Bitwise Left Shift	(10<<2) = 40
>>	Bitwise Right Shift	(10>>2) = 2
>>>	Bitwise Right Shift with Zero	(10>>>2) = 2

JavaScript Logical Operators

The following operators are known as JavaScript logical operators.

Operator	Description	Example
&&	Logical AND	(10==20 && 20==33) = false
	Logical OR	(10==20 20==33) = false
!	Logical Not	!(10==20) = true

JavaScript Assignment Operators

The following operators are known as JavaScript assignment operators.

Operator	Description	Example
=	Assign	10+10 = 20
+=	Add and assign	var a=10; a+=20; Now a = 30
-=	Subtract and assign	var a=20; a-=10; Now a = 10
=	Multiply and assign	var a=10; a=20; Now a = 200
/=	Divide and assign	var a=10; a/=2; Now a = 5
%=	Modulus and assign	var a=10; a%=2; Now a = 0

JavaScript Special Operators

The following operators are known as JavaScript special operators.

Operator	Description
(?:)	Conditional Operator returns value based on the condition. It is like if-else.
,	Comma Operator allows multiple expressions to be evaluated as single statement.

delete Delete Operator deletes a property from the object.
in In Operator checks if object has the given property
instanceof checks if the object is an instance of given type
new creates an instance (object)
typeof checks the type of object.
void it discards the expression's return value.
yield checks what is returned in a generator by the generator's iterator.

Built in functions in JavaScript:

Number Methods

The Number object contains only the default methods that are part of every object's definition.

Sr.No	Method & Description
1	constructor() :Returns the function that created this object's instance. By default this is the Number object.
2	toExponential() :Forces a number to display in exponential notation, even if the number is in the range in which JavaScript normally uses standard notation.
3	toFixed() : Formats a number with a specific number of digits to the right of the decimal.
4	toLocaleString() : Returns a string value version of the current number in a format that may vary according to a browser's locale settings.
5	toPrecision() :Defines how many total digits (including digits to the left and right of the decimal) to display of a number.
6	toString() :Returns the string representation of the number's value.
7	valueOf() :Returns the number's value.

1) JavaScript Number - constructor()

Description

It returns a reference to the Number function that created the instance's prototype.

Syntax:

Its syntax is as follows –

```
number.constructor()
```

Return value

Returns the function that created this object's instance.

Example

```
<html> <head> <title>JavaScript constructor() Method</title> </head>

<body> <script type="text/javascript">

    var num = new Number( 177.1234 );

    document.write("num.constructor() is : " + num.constructor);

</script> </body></html>
```

Output

```
num.constructor() is : function Number() { [native code] }
```

2) JavaScript Number - toExponential()

Description

This method returns a string representing the **number** object in exponential notation.

Syntax

Its syntax is as follows –

```
number.toExponential( [fractionDigits] )
```

Parameter Details

fractionDigits – An integer specifying the number of digits after the decimal point. Defaults to as many digits as necessary to specify the number.

Return Value

A string representing a Number object in exponential notation with one digit before the decimal point, rounded to **fractionDigits** digits after the decimal point. If the **fractionDigits** argument is omitted, the number of digits after the decimal point defaults to the number of digits necessary to represent the value uniquely.

Example

Try the following example.

```

<html>  <head>    <title>Javascript Method toExponential()</title>  </head>

  <body>    <script type="text/javascript">

      var num=77.1234;

      var val = num.toExponential();

      document.write("num.toExponential() is : " + val );

      document.write("<br />");

      val = num.toExponential(4);

      document.write("num.toExponential(4) is : " + val );

      document.write("<br />");

      val = num.toExponential(2);

      document.write("num.toExponential(2) is : " + val);

      document.write("<br />");

      val = 77.1234.toExponential();

      document.write("77.1234.toExponential()is : " + val );

      document.write("<br />");

      val = 77.1234.toExponential();

      document.write("77 .toExponential() is : " + val);

  </script>    </body></html>

```

Output

```

num.toExponential() is : 7.71234e+1
num.toExponential(4) is : 7.7123e+1
num.toExponential(2) is : 7.71e+1
77.1234.toExponential()is:7.71234e+1
77 .toExponential() is : 7.71234e+1

```

3) JavaScript Number - toFixed()

Description

This method formats a **number** with a specific number of digits to the right of the decimal.

Syntax

Its syntax is as follows –

```
number.toFixed( [digits] )
```

Parameter Details

digits – The number of digits to appear after the decimal point.

Return Value

A string representation of number that does not use exponential notation and has the exact number of **digits** after the decimal place.

Example

Try the following example.

```
<html> <head> <title>JavaScript toFixed() Method</title> </head>

<body> <script type="text/javascript">

    var num=177.1234;

    document.write("num.toFixed() is : " + num.toFixed());

    document.write("<br />");

    document.write("num.toFixed(6) is : " + num.toFixed(6));

    document.write("<br />");

    document.write("num.toFixed(1) is : " + num.toFixed(1));

    document.write("<br />");

    document.write("(1.23e+20).toFixed(2) is:" + (1.23e+20).toFixed(2));

    document.write("<br />");

    document.write("(1.23e-10).toFixed(2) is : " + (1.23e-10).toFixed(2));

</script> </body></html>
```

Output

```
num.toFixed() is : 177
num.toFixed(6) is : 177.123400
num.toFixed(1) is : 177.1
(1.23e+20).toFixed(2) is:12300000000000000000.00
(1.23e-10).toFixed(2) is : 0.00
```

4) JavaScript Number - toLocaleString()

Description

This method converts a **number** object into a human readable string representing the number using the locale of the environment.

Syntax

Its syntax is as follows –

```
number.toLocaleString()
```

Return Value

Returns a human readable string representing the number using the locale of the environment.

Example

Try the following example.

```
<html> <head> <title>JavaScript toLocaleString() Method </title> </head>

<body> <script type="text/javascript">

    var num = new Number(177.1234);

    document.write( num.toLocaleString());

</script> </body></html>
```

Output

```
177.1234
```

5) JavaScript Number - toPrecision()

Description

This method returns a string representing the **number** object to the specified precision.

Syntax

Its syntax is as follows –

```
number.toPrecision( [ precision ] )
```

Parameter Details

precision – An integer specifying the number of significant digits.

Return Value

Returns a string representing a Number object in fixed-point or exponential notation rounded **toprecision** significant digits.

Example

Try the following example.

```

<html> <head> <title>JavaScript toPrecision() Method </title> </head>

<body> <script type="text/javascript">

    var num = new Number(7.123456);

    document.write("num.toPrecision() is " + num.toPrecision());

    document.write("<br />");

    document.write("num.toPrecision(4) is " + num.toPrecision(4));

    document.write("<br />");

    document.write("num.toPrecision(2) is " + num.toPrecision(2));

    document.write("<br />");

    document.write("num.toPrecision(1) is " + num.toPrecision(1));

</script> </body></html>

```

Output

```

num.toPrecision() is 7.123456
num.toPrecision(4) is 7.123
num.toPrecision(2) is 7.1
num.toPrecision(1) is 7

```

6) JavaScript Number - toString()

Description

This method returns a string representing the specified object. The **toString()** method parses its first argument, and attempts to return a string representation in the specified radix (base).

Syntax

Its syntax is as follows –

```
number.toString( [radix] )
```

Parameter Details

radix – An integer between 2 and 36 specifying the base to use for representing numeric values.

Return Value

Returns a string representing the specified Number object.

Example

Try the following example.

```

<html> <head> <title>JavaScript toString() Method </title> </head>

```

```

<body>      <script type="text/javascript">

    num = new Number(15);

    document.write("num.toString() is " + num.toString());

    document.write("<br />");

    document.write("num.toString(2) is " + num.toString(2));

    document.write("<br />");

    document.write("num.toString(4) is " + num.toString(4));

    document.write("<br />");

</script>    </body></html>

```

Output

```

num.toString() is 15
num.toString(2) is 1111
num.toString(4) is 33

```

7) JavaScript Number - valueOf()

Description

This method returns the primitive value of the specified **number** object.

Syntax

Its syntax is as follows –

```
number.valueOf()
```

Return Value

Returns the primitive value of the specified **number** object.

Example

Try the following example.

```

<html>  <head>    <title>JavaScript valueOf() Method </title>  </head>

    <body>      <script type="text/javascript">

        var num = new Number(15.11234);

        document.write("num.valueOf() is " + num.valueOf());

    </script>    </body></html>

```

Output

num.valueOf() is 15.11234

Boolean Methods:

Here is a list of each method and its description.

Sr.No	Method & Description
1	toSource() :Returns a string containing the source of the Boolean object; you can use this string to create an equivalent object.
2	toString() : Returns a string of either "true" or "false" depending upon the value of the object.
3	valueOf() : Returns the primitive value of the Boolean object.

1) JavaScript - Boolean toSource() Method

Description

Javascript boolean **toSource()** method returns a string representing the source code of the object.

Note – This method is not compatible with all the browsers.

Syntax

Its syntax is as follows –

```
boolean.toSource()
```

Return Value

Returns a string representing the source code of the object.

Example

Try the following example.

```
<html> <head> <title>JavaScript toSource() Method</title> </head>

<body> <script type="text/javascript">

    function book(title, publisher, price)

    {

        this.title = title;
```

```

    this.publisher = publisher;

    this.price = price;
}

    var newBook = new book("Perl","Leo Inc",200);

    document.write(newBook.toSource());

</script>    </body></html>

```

Output

```
{title:"Perl", publisher:"Leo Inc", price:200}
```

2) JavaScript - Boolean toString() Method

Description

This method returns a string of either "true" or "false" depending upon the value of the object.

Syntax

Its syntax is as follows –

```
boolean.toString()
```

Return Value

Returns a string representing the specified Boolean object.

Example

Try the following example.

```

<html>  <head>    <title>JavaScript toString() Method</title>  </head>

    <body>        <script type="text/javascript">

        var flag = new Boolean(false);

        document.write( "flag.toString is : " + flag.toString() );

    </script>    </body></html>

```

Output: flag.toString is : false

3) JavaScript - Boolean valueOf() Method

Description

Javascript boolean **valueOf()** method returns the primitive value of the specified **boolean** object.

Syntax

Its syntax is as follows –

```
boolean.valueOf()
```

Return Value

Returns the primitive value of the specified **boolean** object.

Example

Try the following example.

```
<html> <head> <title>JavaScript valueOf() Method</title> </head>

<body> <script type="text/javascript">

    var flag = new Boolean(false);

    document.write( "flag.valueOf is : " + flag.valueOf() );

</script> </body></html>
```

Output

```
flag.valueOf is : false
```

String Methods:

Here is a list of each method and its description.

Sr.No	Method & Description
1	charAt() : Returns the character at the specified index.
2	charCodeAt() :Returns a number indicating the Unicode value of the character at the given index.
3	concat() : Combines the text of two strings and returns a new string.
4	indexOf() : Returns the index within the calling String object of the first occurrence of the specified value, or -1 if not found.

5	lastIndexOf(): Returns the index within the calling String object of the last occurrence of the specified value, or -1 if not found.
6	localeCompare(): Returns a number indicating whether a reference string comes before or after or is the same as the given string in sort order.
7	length(): Returns the length of the string.
8	match(): Used to match a regular expression against a string.
9	replace(): Used to find a match between a regular expression and a string, and to replace the matched substring with a new substring.
10	search(): Executes the search for a match between a regular expression and a specified string.
11	slice(): Extracts a section of a string and returns a new string.
12	split(): Splits a String object into an array of strings by separating the string into substrings.
13	substr(): Returns the characters in a string beginning at the specified location through the specified number of characters.
14	substring(): Returns the characters in a string between two indexes into the string.
15	toLocaleLowerCase(): The characters within a string are converted to lower case while respecting the current locale.
16	toLocaleUpperCase(): The characters within a string are converted to upper case while respecting the current locale.
17	toLowerCase(): Returns the calling string value converted to lower case.
18	toString(): Returns a string representing the specified object.
19	toUpperCase(): Returns the calling string value converted to uppercase.
20	valueOf(): Returns the primitive value of the specified object.

1) JavaScript String - charAt() Method

Description

charAt() is a method that returns the character from the specified index.

Characters in a string are indexed from left to right. The index of the first character is 0, and the index of the last character in a string, called **stringName**, is stringName.length – 1.

Syntax

Use the following syntax to find the character at a particular index.

```
string.charAt(index);
```

Argument Details

index – An integer between 0 and 1 less than the length of the string.

Return Value

Returns the character from the specified index.

Example

Try the following example.

```
<html> <head> <title>JavaScript String charAt() Method</title> </head>

<body> <script type="text/javascript">

    var str = new String( "This is string" );

    document.writeln("str.charAt(0) is:" + str.charAt(0));

    document.writeln("<br />str.charAt(1) is:" + str.charAt(1));

    document.writeln("<br />str.charAt(2) is:" + str.charAt(2));

    document.writeln("<br />str.charAt(3) is:" + str.charAt(3));

    document.writeln("<br />str.charAt(4) is:" + str.charAt(4));

    document.writeln("<br />str.charAt(5) is:" + str.charAt(5));

</script> </body></html>
```

Output

```
str.charAt(0) is:T
str.charAt(1) is:h
str.charAt(2) is:i
str.charAt(3) is:s
str.charAt(4) is:
str.charAt(5) is:i
```

2) JavaScript String - concat() Method

Description

This method adds two or more strings and returns a new single string.

Syntax

Its syntax is as follows –

```
string.concat(string2, string3[, ..., stringN]);
```

Argument Details

string2...stringN – These are the strings to be concatenated.

Return Value

Returns a single concatenated string.

Example

Try the following example.

```
<html> <head> <title>JavaScript String concat() Method</title> </head>

<body> <script type="text/javascript">

    var str1 = new String( "This is string one" );

    var str2 = new String( "This is string two" );

    var str3 = str1.concat( str2 );

    document.write("Concatenated String :" + str3);

</script> </body></html>
```

Output

Concatenated String :This is string oneThis is string two.

3) JavaScript String - length Property

Description

This property returns the number of characters in a string.

Syntax

Use the following syntax to find the length of a string –

```
string.length
```

Return Value

Returns the number of characters in the string.

Example

Try the following example.

```
<html> <head> <title>JavaScript String length Property</title> </head>

<body> <script type="text/javascript">

    var str = new String( "This is string" );

    document.write("str.length is:" + str.length);

</script> </body></html>
```

Output

```
str.length is:14
```

4) JavaScript String - toLowerCase() Method

Description

This method returns the calling string value converted to lowercase.

Syntax

Its syntax is as follows –

```
string.toLowerCase( )
```

Return Value

Returns the calling string value converted to lowercase.

Example

Try the following example.

```
<html> <head> <title>JavaScript String toLowerCase() Method</title> </head>

<body> <script type="text/javascript">

    var str = "Apples are round, and Apples are Juicy.";

    document.write(str.toLowerCase( ));

</script> </body></html>
```

Output

```
apples are round, and apples are juicy.
```

5) JavaScript String - toString() Method

Description

This method returns a string representing the specified object.

Syntax

Its syntax is as follows –

```
string.toString( )
```

Return Value

Returns a string representing the specified object.

Example

Try the following example.

```
<html> <head> <title>JavaScript String toString() Method</title> </head>

<body> <script type="text/javascript">

    var str = "Apples are round, and Apples are Juicy.";

    document.write(str.toString( ));

</script> </body></html>
```

Output

Apples are round, and Apples are Juicy.

6) JavaScript String - toUpperCase() Method

Description

This method returns the calling string value converted to uppercase.

Syntax

Its syntax is as follows –

```
string.toUpperCase( )
```

Return Value

Returns a string representing the specified object.

Example

Try the following example.

```
<html> <head> <title>JavaScript String toUpperCase() Method</title> </head>

<body> <script type="text/javascript">
```

```
var str = "Apples are round, and Apples are Juicy.";

document.write(str.toUpperCase( ));

</script> </body></html>
```

Output

APPLES ARE ROUND, AND APPLES ARE JUICY.

7) JavaScript String - valueOf() Method

Description

This method returns the primitive value of a String object.

Syntax

Its syntax is as follows –

```
string.valueOf( )
```

Return Value

Returns the primitive value of a String object.

Example

Try the following example.

```
<html> <head> <title>JavaScript String valueOf() Method</title> </head>

<body> <script type="text/javascript">

var str = new String("Hello world");

document.write(str.valueOf( ));

</script> </body></html>
```

Output: Hello world

String HTML wrappers

Here is a list of each method which returns a copy of the string wrapped inside the appropriate HTML tag.

Sr.No	Method & Description
1	anchor() :Creates an HTML anchor that is used as a hypertext target.

2	big() :Creates a string to be displayed in a big font as if it were in a <big> tag.
3	blink() : Creates a string to blink as if it were in a <blink> tag.
4	bold() : Creates a string to be displayed as bold as if it were in a tag.
5	fixed() : Causes a string to be displayed in fixed-pitch font as if it were in a <tt> tag
6	fontcolor() :Causes a string to be displayed in the specified color as if it were in a tag.
7	fontsize() : Causes a string to be displayed in the specified font size as if it were in a tag.
8	italics() : Causes a string to be italic, as if it were in an <i> tag.
9	link() : Creates an HTML hypertext link that requests another URL.
10	small() :Causes a string to be displayed in a small font, as if it were in a <small> tag.
11	strike() : Causes a string to be displayed as struck-out text, as if it were in a <strike> tag.
12	sub() : Causes a string to be displayed as a subscript, as if it were in a <sub> tag
13	sup() :Causes a string to be displayed as a superscript, as if it were in a <sup> tag

1) JavaScript String - anchor() Method

Description

This method creates an HTML anchor that is used as a hypertext target.

Syntax

Its syntax is as follows –

```
string.anchor( anchorname )
```

Attribute details

anchorname – Defines a name for the anchor.

Return Value

Returns the string having the anchor tag.

Example

Try the following example.

```
<html> <head> <title>JavaScript String anchor() Method</title> </head>

<body> <script type="text/javascript">

    var str = new String("Hello world");

    alert(str.anchor( "myanchor" ));

</script> </body></html>
```

Output

```
<a name="myanchor">Hello world</a>
```

2) JavaScript String - big() Method

Description

This method causes a string to be displayed in a big font as if it were in a BIG tag.

Syntax

The syntax to use big() is as follows –

```
string.big( )
```

Return Value

Returns the string having **<big>** tag.

Example

Try the following example.

```
<html> <head> <title>JavaScript String big() Method</title> </head>

<body> <script type="text/javascript">

    var str = new String("Hello world");

    alert(str.big());

</script> </body></html>
```

Output: <big>Hello world</big>

3) JavaScript String - blink() Method

Description

This method causes a string to blink as if it were in a BLINK tag.

Syntax

The syntax for blink() method is as follows –

```
string.blink( )
```

Return Value

Returns the string with **<blink>** tag.

Example

Try the following example.

```
<html> <head> <title>JavaScript String blink() Method</title> </head>

<body> <script type="text/javascript">

    var str = new String("Hello world");

    alert(str.blink());

</script> </body></html>
```

Output: <blink>Hello world</blink>

4) JavaScript String - bold() Method

Description

This method causes a string to be displayed as bold as if it were in a **** tag.

Syntax

The syntax for bold() method is as follows –

```
string.bold()
```

Return Value

Returns the string with **<bold>** tag.

Example

Try the following example.

```
<html> <head> <title>JavaScript String bold() Method</title> </head>

<body> <script type="text/javascript">

    var str = new String("Hello world");

    alert(str.bold());

</script> </body></html>
```

Output : Hello world

Array Methods

Here is a list of each method and its description.

Sr.No	Method & Description
1	concat() : Returns a new array comprised of this array joined with other array(s) and/or value(s).
2	every() : Returns true if every element in this array satisfies the provided testing function.
3	filter() : Creates a new array with all of the elements of this array for which the provided filtering function returns true.
4	forEach() : Calls a function for each element in the array.
5	indexOf() : Returns the first (least) index of an element within the array equal to the specified value, or -1 if none is found.
6	join() : Joins all elements of an array into a string.
7	lastIndexOf() : Returns the last (greatest) index of an element within the array equal to the specified value, or -1 if none is found.
8	map() : Creates a new array with the results of calling a provided function on every element in this array.
9	pop() : Removes the last element from an array and returns that element.
10	push() : Adds one or more elements to the end of an array and returns the new length of the array.
11	reduce() : Apply a function simultaneously against two values of the array (from left-to-right) as to reduce it to a single value.
12	reduceRight() : Apply a function simultaneously against two values of the array (from right-to-left) as to reduce it to a single value.
13	reverse() : Reverses the order of the elements of an array -- the first becomes the last, and the last becomes the first.
14	shift() : Removes the first element from an array and returns that element.
15	slice() : Extracts a section of an array and returns a new array.
16	some() : Returns true if at least one element in this array satisfies the provided testing function.
17	toSource() : Represents the source code of an object

18	sort() :Sorts the elements of an array.
19	splice() :Adds and/or removes elements from an array.
20	toString() :Returns a string representing the array and its elements.
21	unshift() : Adds one or more elements to the front of an array and returns the new length of the array.

1) JavaScript - Array concat() Method

Description

Javascript array **concat()** method returns a new array comprised of this array joined with two or more arrays.

Syntax

The syntax of concat() method is as follows –

```
array.concat(value1, value2, ..., valueN);
```

valueN – Arrays and/or values to concatenate to the resulting array.

Return Value

Returns the length of the array.

Example

Try the following example.

```
<html> <head> <title>JavaScript Array concat Method</title> </head>

<body> <script type="text/javascript">

    var alpha = ["a", "b", "c"];

    var numeric = [1, 2, 3];

    var alphaNumeric = alpha.concat(numeric);

    document.write("alphaNumeric : " + alphaNumeric );

</script> </body></html>
```

Output: alphaNumeric : a,b,c,1,2,3

2) JavaScript - Array join() Method

Description

Javascript array **join()** method joins all the elements of an array into a string.

Syntax

Its syntax is as follows –

```
array.join(separator);
```

Parameter Details

separator – Specifies a string to separate each element of the array. If omitted, the array elements are separated with a comma.

Return Value

Returns a string after joining all the array elements.

Example

Try the following example.

```
<html> <head> <title>JavaScript Array join Method</title> </head>

<body> <script type="text/javascript">

    var arr = new Array("First","Second","Third");

    var str = arr.join();

    document.write("str : " + str );

    var str = arr.join(", ");

    document.write("<br />str : " + str );

    var str = arr.join(" + ");

    document.write("<br />str : " + str );

</script> </body></html>
```

Output

```
str : First,Second,Third
str : First, Second, Third
str : First + Second + Third
```

3) JavaScript - Array pop() Method

Description

Javascript array **pop()** method removes the last element from an array and returns that element.

Syntax

Its syntax is as follows –

```
array.pop();
```

Return Value

Returns the removed element from the array.

Example

Try the following example.

```
<html> <head> <title>JavaScript Array pop Method</title> </head>

<body> <script type="text/javascript">

    var numbers = [1, 4, 9];

    var element = numbers.pop();

    document.write("element is : " + element );

    var element = numbers.pop();

    document.write("<br />element is : " + element );

</script> </body></html>
```

Output

```
element is : 9
element is : 4
```

4) JavaScript - Array push() Method

Description

Javascript array **push()** method appends the given element(s) in the last of the array and returns the length of the new array.

Syntax

Its syntax is as follows –

```
array.push(element1, ..., elementN);
```

Parameter Details

element1, ..., elementN: The elements to add to the end of the array.

Return Value

Returns the length of the new array.

Example

Try the following example.

```
<html> <head> <title>JavaScript Array push Method</title> </head>

<body> <script type="text/javascript">
```

```

var numbers = new Array(1, 4, 9);

var length = numbers.push(10);

document.write("new numbers is : " + numbers );

length = numbers.push(20);

document.write("<br />new numbers is : " + numbers );

</script>    </body></html>

```

Output

```

new numbers is : 1,4,9,10
new numbers is : 1,4,9,10,20

```

5) JavaScript Array reverse() Method

Description

Javascript array **reverse()** method reverses the element of an array. The first array element becomes the last and the last becomes the first.

Syntax

Its syntax is as follows –

```
array.reverse();
```

Return Value

Returns the reversed single value of the array.

Example

Try the following example.

```

<html>  <head>    <title>JavaScript Array reverse Method</title>  </head>

  <body>    <script type="text/javascript">

      var arr = [0, 1, 2, 3].reverse();

      document.write("Reversed array is : " + arr );

  </script>    </body></html>

```

Output:: Reversed array is : 3,2,1,0

6) JavaScript - Array sort() Method

Description

Javascript array **sort()** method sorts the elements of an array.

Syntax

Its syntax is as follows –

```
array.sort( compareFunction );
```

Parameter Details

compareFunction – Specifies a function that defines the sort order. If omitted, the array is sorted lexicographically.

Return Value

Returns a sorted array.

Example

Try the following example.

```
<html> <head> <title>JavaScript Array sort Method</title> </head>

<body> <script type="text/javascript">

    var arr = new Array("orange", "mango", "banana", "sugar");

    var sorted = arr.sort();

    document.write("Returned string is : " + sorted );

</script> </body></html>
```

Output:: Returned array is : banana,mango,orange,sugar

Date Methods

Here is a list of each method and its description.

Sr.No	Method & Description
1	Date() :Returns today's date and time
2	getDate() :Returns the day of the month for the specified date according to local time.
3	getDay() : Returns the day of the week for the specified date according to local time.
4	getFullYear() :Returns the year of the specified date according to local time.
5	getHours() :Returns the hour in the specified date according to local time.

6	getMilliseconds() :Returns the milliseconds in the specified date according to local time.
7	getMinutes() : Returns the minutes in the specified date according to local time.
8	getMonth() :Returns the month in the specified date according to local time.
9	getSeconds() :Returns the seconds in the specified date according to local time.
10	getTime() : Returns the numeric value of the specified date as the number of milliseconds since January 1, 1970, 00:00:00 UTC.
11	getTimezoneOffset() :Returns the time-zone offset in minutes for the current locale.
12	getUTCDate() :Returns the day (date) of the month in the specified date according to universal time.
13	getUTCDay() :Returns the day of the week in the specified date according to universal time.
14	getUTCFullYear() :Returns the year in the specified date according to universal time.
15	getUTCHours() :Returns the hours in the specified date according to universal time.
16	getUTCMilliseconds() :Returns the milliseconds in the specified date according to universal time.
17	getUTCMinutes() :Returns the minutes in the specified date according to universal time.
18	getUTCMonth() :Returns the month in the specified date according to universal time.
19	getUTCSeconds() :Returns the seconds in the specified date according to universal time.
20	getYear() : Deprecated - Returns the year in the specified date according to local time. Use getFullYear instead.
21	setDate() :Sets the day of the month for a specified date according to local time.
22	setFullYear() :Sets the full year for a specified date according to local time.
23	setHours() :Sets the hours for a specified date according to local time.
24	setMilliseconds() :Sets the milliseconds for a specified date according to local time.
25	setMinutes() :Sets the minutes for a specified date according to local time.
26	setMonth() :Sets the month for a specified date according to local time.

27	setSeconds() :Sets the seconds for a specified date according to local time.
28	setTime() :Sets the Date object to the time represented by a number of milliseconds since January 1, 1970, 00:00:00 UTC.
29	setUTCDate() :Sets the day of the month for a specified date according to universal time.
30	setUTCFullYear() :Sets the full year for a specified date according to universal time.
31	setUTCHours() : Sets the hour for a specified date according to universal time.
32	setUTCMilliseconds() :Sets the milliseconds for a specified date according to universal time.
33	setUTCMinutes() : Sets the minutes for a specified date according to universal time.
34	setUTCMonth() :Sets the month for a specified date according to universal time.
35	setUTCSeconds() :Sets the seconds for a specified date according to universal time.
36	setYear() : Deprecated - Sets the year for a specified date according to local time. Use setFullYear instead.
37	toDateString() : Returns the "date" portion of the Date as a human-readable string.
38	toGMTString() ::: Deprecated - Converts a date to a string, using the Internet GMT conventions. Use toUTCString instead.
39	toLocaleDateString() : Returns the "date" portion of the Date as a string, using the current locale's conventions.
40	toLocaleFormat() :Converts a date to a string, using a format string.
41	toLocaleString() :Converts a date to a string, using the current locale's conventions.
42	toLocaleTimeString() :Returns the "time" portion of the Date as a string, using the current locale's conventions.
43	toSource() :Returns a string representing the source for an equivalent Date object; you can use this value to create a new object.
44	toString() :Returns a string representing the specified Date object.
45	toTimeString() :Returns the "time" portion of the Date as a human-readable string.
46	toUTCString() :Converts a date to a string, using the universal time convention.

47

valueOf():Returns the primitive value of a Date object.**Date Static Methods**

In addition to the many instance methods listed previously, the Date object also defines two static methods. These methods are invoked through the Date() constructor itself –

Sr.No	Method & Description
1	Date.parse(): Parses a string representation of a date and time and returns the internal millisecond representation of that date.
2	Date.UTC(): Returns the millisecond representation of the specified UTC date and time.

1) JavaScript - Date() Method**Description**

Javascript **Date()** method returns today's date and time and does not need any object to be called.

Syntax

Its syntax is as follows –

Date()

Return Value

Returns today's date and time.

Example

Try the following example.

```
<html> <head> <title>JavaScript Date Method</title> </head>

<body> <script type="text/javascript">

    var dt = Date();

    document.write("Date and Time : " + dt );

</script> </body> </html>
```

Output

Date and Time : Thu Mar 01 2018 02:01:00 GMT-0800 (Pacific Standard Time)

2) JavaScript - Date getDate() Method

Description

Javascript date **getDate()** method returns the day of the month for the specified date according to local time. The value returned by **getDate()** is an integer between 1 and 31.

Syntax

Its syntax is as follows –

```
Date.getDate()
```

Return Value

Returns today's date and time.

Example

```
<html> <head> <title>JavaScript getDate Method</title> </head>

<body>

<script type="text/javascript">

var dt = new Date("December 25, 1995 23:15:00");

document.write("getDate() : " + dt.getDate() );

</script> </body></html>
```

Output

getDate() : 25

3) JavaScript - Date getDay() Method

Description

Javascript date **getDay()** method returns the day of the week for the specified date according to local time. The value returned by **getDay()** is an integer corresponding to the day of the week: 0 for Sunday, 1 for Monday, 2 for Tuesday, and so on.

Syntax

Its syntax is as follows –

```
Date.getDay()
```

Return Value

Returns the day of the week for the specified date according to local time.

Example

Try the following example.

```
<html> <head> <title>JavaScript getDay Method</title> </head>

<body> <script type="text/javascript">

    var dt = new Date("December 25, 1995 23:15:00");

    document.write("getDay() : " + dt.getDay() );

</script> </body></html>
```

Output

```
getDay() : 1
```

4) JavaScript - Date getFullYear() Method

Description

Javascript date **getFullYear()** method returns the year of the specified date according to local time. The value returned by **getFullYear()** is an absolute number. For dates between the years 1000 and 9999, **getFullYear()** returns a four-digit number, for example, 2008.

Syntax

Its syntax is as follows –

```
Date.getFullYear()
```

Return Value

Returns the year of the specified date according to local time.

Example

Try the following example.

```
<html> <head> <title>JavaScript getFullYear Method</title> </head>

<body> <script type="text/javascript">

    var dt = new Date("December 25, 1995 23:15:00");

    document.write("getFullYear() : " + dt.getFullYear() );

</script> </body></html>
```

Output

```
getFullYear() : 1995
```

5) JavaScript - Date getHours() Method

Description

Javascript Date **getHours()** method returns the hour in the specified date according to local time. The value returned by **getHours()** is an integer between 0 and 23.

Syntax

Its syntax is as follows –

```
Date.getHours()
```

Return Value

Returns the hour in the specified date according to local time.

Example

Try the following example.

```
<html>  <head>    <title>JavaScript getHours Method</title>  </head>

  <body>    <script type="text/javascript">

      var dt = new Date("December 25, 1995 23:15:00");

      document.write("getHours() : " + dt.getHours() );

    </script>  </body></html>
```

Output: getHours() : 23

6) JavaScript - Date getMilliseconds() Method

Description

Javascript date **getMilliseconds()** method returns the milliseconds in the specified date according to local time. The value returned by **getMilliseconds()** is a number between 0 and 999.

Syntax

Its syntax is as follows –

```
Date.getMilliseconds()
```

Return Value

Returns the milliseconds in the specified date according to local time.

Example

Try the following example.

```
<html>  <head>    <title>JavaScript getMilliseconds Method</title>  </head>

  <body>    <script type="text/javascript">

      var dt = new Date( );
```

```
document.write("getMilliseconds() : " + dt.getMilliseconds() );  
  
</script> </body></html>
```

Output: getMilliseconds() : 676

7) JavaScript - Date getMinutes() Method

Description

Javascript date **getMinutes()** method returns the minutes in the specified date according to local time. The value returned by **getMinutes()** is an integer between 0 and 59.

Syntax

Its syntax is as follows –

```
Date.getMinutes()
```

Return Value

Returns the minutes in the specified date according to local time.

Example

Try the following example.

```
<html> <head> <title>JavaScript getMinutes Method</title> </head>  
  
<body> <script type="text/javascript">  
  
var dt = new Date( "December 25, 1995 23:15:00" );  
  
document.write("getMinutes() : " + dt.getMinutes() );  
  
</script> </body></html>
```

Output: getMinutes() : 15

Math Methods

Here is a list of each method and its description.

Sr.No	Method & Description
1	abs() :Returns the absolute value of a number.
2	acos() : Returns the arccosine (in radians) of a number.

3	asin() : Returns the arcsine (in radians) of a number.
4	atan() : Returns the arctangent (in radians) of a number.
5	atan2() : Returns the arctangent of the quotient of its arguments.
6	ceil() : Returns the smallest integer greater than or equal to a number.
7	cos() : Returns the cosine of a number.
8	exp() :Returns E^N , where N is the argument, and E is Euler's constant, the base of the natural logarithm.
9	floor() : Returns the largest integer less than or equal to a number.
10	log() : Returns the natural logarithm (base E) of a number.
11	max() :Returns the largest of zero or more numbers.
12	min() :Returns the smallest of zero or more numbers.
13	pow() :Returns base to the exponent power, that is, base exponent.
14	random() : Returns a pseudo-random number between 0 and 1.
15	round() :Returns the value of a number rounded to the nearest integer.
16	sin() :Returns the sine of a number.
17	sqrt() :Returns the square root of a number.
18	tan() :Returns the tangent of a number.
19	toSource() : Returns the string "Math".

1) JavaScript - Math abs Method

Description

This method returns the absolute value of a number.

Syntax

Its syntax is as follows –

```
Math.abs( x );
```

Parameter Details

x – A number.

Return Value

Returns the absolute value of a number.

Example

Try the following example program.

```
<html> <head> <title>JavaScript Math abs() Method</title> </head>

<body> <script type="text/javascript">

    var value = Math.abs(-1);

    document.write("First Test Value : " + value );

    var value = Math.abs(null);

    document.write("<br />Second Test Value : " + value );

    var value = Math.abs(20);

    document.write("<br />Third Test Value : " + value );

    var value = Math.abs("string");

    document.write("<br />Fourth Test Value : " + value );

</script> </body></html>
```

Output

```
First Test Value : 1
Second Test Value : 0
Third Test Value : 20
Fourth Test Value : NaN
```

2) JavaScript - Math log Method

Description

This method returns the natural logarithm (base E) of a number. If the value of number is negative, the return value is always **NaN**.

Syntax

Its syntax is as follows –

```
Math.log( x );
```

Parameter Details

x – A numbers.

Return Value

Returns the natural logarithm (base E) of a number.

Example

Try the following example program.

```
<html> <head> <title>JavaScript Math log() Method</title> </head>

<body> <script type="text/javascript">

    var value = Math.log(10);

    document.write("First Test Value : " + value );

    var value = Math.log(0);

    document.write("<br />Second Test Value : " + value );

    var value = Math.log(-1);

    document.write("<br />Third Test Value : " + value );

    var value = Math.log(100);

    document.write("<br />Fourth Test Value : " + value );

</script> </body></html>
```

Output

```
First Test Value : 2.302585092994046
Second Test Value : -Infinity
Third Test Value : NaN
Fourth Test Value : 4.605170185988092
```

3) JavaScript - Math max Method

Description

This method returns the largest of zero or more numbers. If no arguments are given, the results is **-Infinity**.

Syntax

Its syntax is as follows –

```
Math.max(value1, value2, ... valueN );
```

Parameter Details

value1, value2, ... valueN : Numbers.

Return Value

Returns the largest of zero or more numbers.

Example

Try the following example program.

```
<html>  <head>    <title>JavaScript Math max() Method</title>  </head>

  <body>      <script type="text/javascript">

    var value = Math.max(10, 20, -1, 100);

    document.write("First Test Value : " + value );

    var value = Math.max(-1, -3, -40);

    document.write("<br />Second Test Value : " + value );

    var value = Math.max(0, -1);

    document.write("<br />Third Test Value : " + value );

    var value = Math.max(100);

    document.write("<br />Fourth Test Value : " + value );

  </script>  </body></html>
```

Output

```
First Test Value : 100
Second Test Value : -1
Third Test Value : 0
Fourth Test Value : 100
```

4) JavaScript - Math min Method

Description

This method returns the smallest of zero or more numbers. If no arguments are given, the results is **+Infinity**.

Syntax

Its syntax is as follows –

```
Math.min(value1, value2, ... valueN ) ;
```

Parameter Details

value1, value2, ... valueN : Numbers.

Return Value

Returns the smallest of zero or more numbers.

Example

```
<html> <head> <title>JavaScript Math min() Method</title> </head>

<body> <script type="text/javascript">

    var value = Math.min(10, 20, -1, 100);

    document.write("First Test Value : " + value );

    var value = Math.min(-1, -3, -40);

    document.write("<br />Second Test Value : " + value );

    var value = Math.min(0, -1);

    document.write("<br />Third Test Value : " + value );

    var value = Math.min(100);

    document.write("<br />Fourth Test Value : " + value );

</script> </body></html>
```

Output

```
First Test Value : -1
Second Test Value : -40
Third Test Value : -1
Fourth Test Value : 100
```

5) JavaScript - Math pow Method

Description

This method returns the base to the exponent power, that is, $\text{base}^{\text{exponent}}$.

Syntax

Its syntax is as follows –

```
Math.pow(base, exponent );
```

Parameter Details

- **base** – The base number.

- **exponents** – The exponent to which to raise base.

Return Value

Returns the base to the exponent power, that is, $\text{base}^{\text{exponent}}$.

Example

Try the following example program.

```
<html> <head> <title>JavaScript Math pow() Method</title> </head>

<body> <script type="text/javascript">

    var value = Math.pow(7, 2);

    document.write("First Test Value : " + value );

    var value = Math.pow(8, 8);

    document.write("<br />Second Test Value : " + value );

    var value = Math.pow(-1, 2);

    document.write("<br />Third Test Value : " + value );

    var value = Math.pow(0, 10);

    document.write("<br />Fourth Test Value : " + value );

</script> </body></html>
```

Output

```
First Test Value : 49
Second Test Value : 16777216
Third Test Value : 1
Fourth Test Value : 0
```

6) JavaScript - Math random Method

Description

This method returns a random number between 0 (inclusive) and 1 (exclusive).

Syntax

Its syntax is as follows –

```
Math.random() ;
```

Return Value

Returns a random number between 0 (inclusive) and 1 (exclusive).

Example

```

<html>  <head>    <title>JavaScript Math random() Method</title>  </head>

  <body>      <script type="text/javascript">

    var value = Math.random( );

    document.write("First Test Value : " + value );

    var value = Math.random( );

    document.write("<br />Second Test Value : " + value );

    var value = Math.random( );

    document.write("<br />Third Test Value : " + value );

    var value = Math.random( );

    document.write("<br />Fourth Test Value : " + value );

  </script>    </body></html>

```

Output

First Test Value : 0.47902729518250986
 Second Test Value : 0.2155848144225727
 Third Test Value : 0.10909082017928995
 Fourth Test Value : 0.5796075247216124

7) JavaScript - Math sqrt Method

Description

This method returns the square root of a number. If the value of a number is negative, sqrt returns NaN.

Syntax

Its syntax is as follows –

```
Math.sqrt( x );
```

Parameter Details

x – A number

Return Value

Returns the square root of a given number.

Example

```

<html>  <head>    <title>JavaScript Math sqrt() Method</title>  </head>

```

```

<body>      <script type="text/javascript">

    var value = Math.sqrt( 0.5 );

    document.write("First Test Value : " + value );

    var value = Math.sqrt( 81 );

    document.write("<br />Second Test Value : " + value );

    var value = Math.sqrt( 13 );

    document.write("<br />Third Test Value : " + value );

    var value = Math.sqrt( -4 );

    document.write("<br />Fourth Test Value : " + value );

</script>  </body></html>

```

Output

```

First Test Value : 0.7071067811865476
Second Test Value : 9
Third Test Value : 3.605551275463989
Fourth Test Value : NaN

```

RegExp Methods

Here is a list of each method and its description.

Sr.No	Method & Description
1	exec() : Executes a search for a match in its string parameter.
2	test() : Tests for a match in its string parameter.
3	toSource() : Returns an object literal representing the specified object; you can use this value to create a new object.
4	toString() :Returns a string representing the specified object.

Control structure in JavaScript:

JavaScript If-else

The **JavaScript if-else statement** is used to *execute the code whether condition is true or false*. There are three forms of if statement in JavaScript.

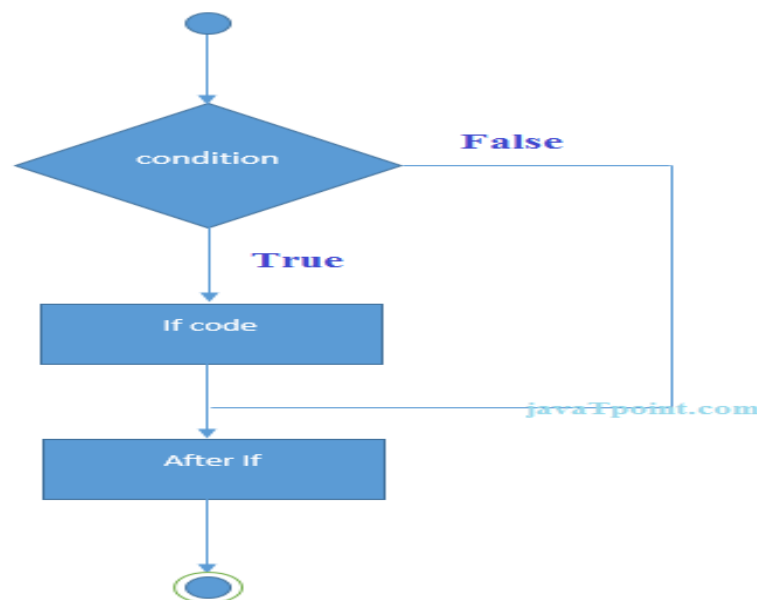
1. If Statement
2. If else statement
3. if else if statement

JavaScript If statement

It evaluates the content only if expression is true. The signature of JavaScript if statement is given below.

1. `if(expression){`
2. `//content to be evaluated`
3. `}`

Flowchart of JavaScript If statement



Let's see the simple example of if statement in javascript.

1. `<script>`
2. `var a=20;`
3. `if(a>10){`
4. `document.write("value of a is greater than 10");`
5. `}`
6. `</script>`

Output of the above example

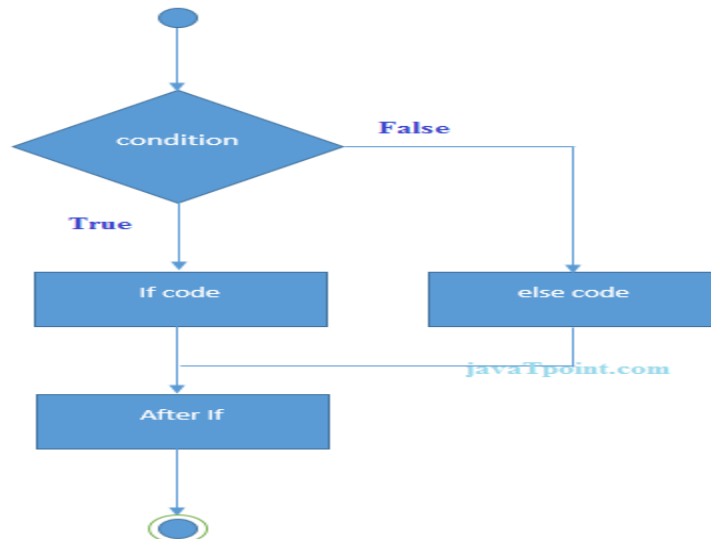
value of a is greater than 10

JavaScript If...else Statement

It evaluates the content whether condition is true or false. The syntax of JavaScript if-else statement is given below.

```
if(expression){  
    //content to be evaluated if condition is true  
}  
else{  
    //content to be evaluated if condition is false  
}
```

Flowchart of JavaScript If...else statement



Let's see the example of if-else statement in JavaScript to find out the even or odd number.

```
<script>
var a=20;
if(a%2==0){
document.write("a is even number");
}
else{
document.write("a is odd number");
}
</script>
```

Output of the above example

a is even number

JavaScript If...else if statement

It evaluates the content only if expression is true from several expressions. The signature of JavaScript if else if statement is given below.

```
if(expression1){
//content to be evaluated if expression1 is true
}
else if(expression2){
//content to be evaluated if expression2 is true
}
else if(expression3){
//content to be evaluated if expression3 is true
}
else{
//content to be evaluated if no expression is true
}
```

Let's see the simple example of if else if statement in javascript.

```
<script>
var a=20;
```

```

if(a==10){
document.write("a is equal to 10");
}
else if(a==15){
document.write("a is equal to 15");
}
else if(a==20){
document.write("a is equal to 20");
}
else{
document.write("a is not equal to 10, 15 or 20");
}
</script>

```

Output of the above example

a is equal to 20

JavaScript Switch

The **JavaScript switch statement** is used *to execute one code from multiple expressions*. It is just like else if statement that we have learned in previous page. But it is convenient than *if..else..if* because it can be used with numbers, characters etc.

The signature of JavaScript switch statement is given below.

```

switch(expression){
    case value1:
        code to be executed;
        break;
    case value2:
        code to be executed;
        break;
    .....

    default:
        code to be executed if above values are not matched;
}

```

Let's see the simple example of switch statement in javascript.

```

<script>
var grade='B';
var result;
switch(grade){
case 'A':
result="A Grade";
break;
case 'B':
result="B Grade";
break;
case 'C':
result="C Grade";
break;
default:
result="No Grade";
}

```

```
}  
document.write(result);  
</script>
```

Output of the above example

B Grade

The switch statement is fall-through i.e. all the cases will be evaluated if you don't use break statement.

Let's understand the behaviour of switch statement in JavaScript.

```
<script>  
var grade='B';  
var result;  
switch(grade){  
case 'A':  
result+=" A Grade";  
case 'B':  
result+=" B Grade";  
case 'C':  
result+=" C Grade";  
default:  
result+=" No Grade";  
}  
document.write(result);  
</script>
```

Output of the above example

B Grade B Grade C Grade No Grade

JavaScript Loops

The **JavaScript loops** are used *to iterate the piece of code* using for, while, do while or for-in loops. It makes the code compact. It is mostly used in array.

There are four types of loops in JavaScript.

1. for loop
2. while loop
3. do-while loop
4. for-in loop

1) JavaScript For loop

The **JavaScript for loop** *iterates the elements for the fixed number of times*. It should be used if number of iteration is known. The syntax of for loop is given below.

```
for (initialization; condition; increment)  
{  
    code to be executed  
}
```

Let's see the simple example of for loop in javascript.

```
<script>  
for (i=1; i<=5; i++)  
{  
    document.write(i + "<br/>")  
}  
</script>
```

Output:

1
2

3
4
5

2) JavaScript while loop

The **JavaScript while loop** *iterates the elements for the infinite number of times*. It should be used if number of iteration is not known. The syntax of while loop is given below.

```
while (condition)
{
    code to be executed
}
```

Let's see the simple example of while loop in javascript.

```
<script>
var i=11;
while (i<=15)
{
    document.write(i + "<br/>");
    i++;
}
</script>
```

Output:

11
12
13
14
15

3) JavaScript do while loop

The **JavaScript do while loop** *iterates the elements for the infinite number of times* like while loop. But, code is *executed at least* once whether condition is true or false. The syntax of do while loop is given below.

```
do{
    code to be executed
}while (condition);
```

Let's see the simple example of do while loop in javascript.

```
<script>
var i=21;
do{
    document.write(i + "<br/>");
    i++;
}while (i<=25);
</script>
```

Output:

21
22
23
24
25

4) JavaScript for in loop

The **JavaScript for in loop** is used *to iterate the properties of an object*. We will discuss about it later.

Document Object Model :

1. [Document Object](#)
2. [Properties of document object](#)
3. [Methods of document object](#)
4. [Example of document object](#)

The **document object** represents the whole html document.

When html document is loaded in the browser, it becomes a document object. It is the **root element** that represents the html document.

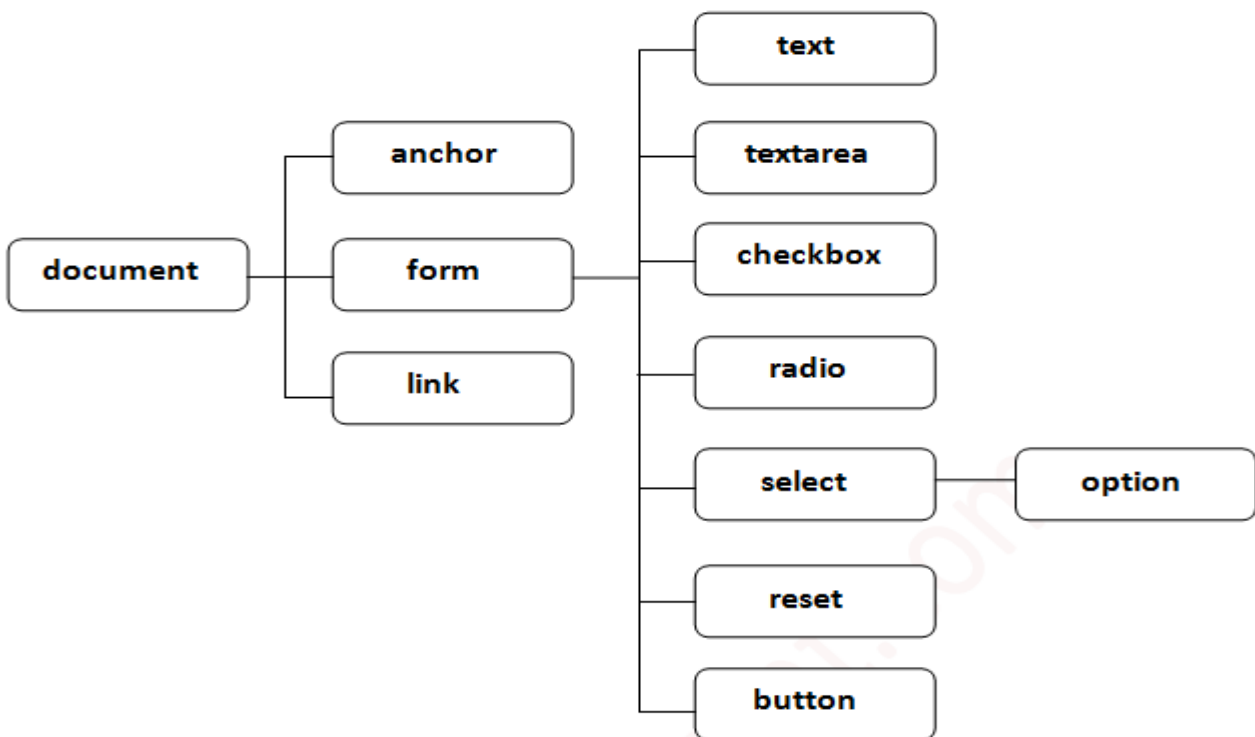
As mentioned earlier, it is the object of window. So
window.document

Is same as
document

According to W3C - *"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."*

Properties of document object

Let's see the properties of document object that can be accessed and modified by the document object.



Methods of document object

We can access and change the contents of document by its methods.

The important methods of document object are as follows:

Method

Description

<code>write("string")</code>	writes the given string on the document.
<code>writeln("string")</code>	writes the given string on the document with newline character at the end.
<code>getElementById()</code>	returns the element having the given id value.
<code>getElementsByName()</code>	returns all the elements having the given name value.
<code>getElementsByTagName()</code>	returns all the elements having the given tag name.
<code>getElementsByClassName()</code>	returns all the elements having the given class name.

Accessing the field value by document object

In this example, we are going to get the value of input text by user. Here, we are using **document.form1.name.value** to get the value of name field.

Here, **document** is the root element that represents the html document.

form1 is the name of the form.

name is the attribute name of the input text.

value is the property, that returns the value of the input text.

Let's see the simple example of document object that prints name with welcome message.

```
<script type="text/javascript">
function printvalue(){
    var name=document.form1.name.value;
    alert("Welcome: "+name);
}
</script>

<form name="form1">
Enter Name:<input type="text" name="name"/>
<input type="button" onclick="printvalue()" value="print name"/>
</form>
```

Output of the above example

Enter Name:

document.getElementById() method

The **document.getElementById()** method returns the element of specified id.

In the previous page, we have used **document.form1.name.value** to get the value of the input value. Instead of this, we can use **document.getElementById()** method to get value of the input text. But we need to define id for the input field.

Let's see the simple example of **document.getElementById()** method that prints cube of the given number.

```
<script type="text/javascript">
function getcube(){
    var number=document.getElementById("number").value;
    alert(number*number*number);
}
```

```
</script>
<form>
Enter No:<input type="text" id="number" name="number"/><br/>
<input type="button" value="cube" onclick="getcube()"/>
</form>
```

Output of the above example

EnterNo:

document.getElementsByName()

The **document.getElementsByName()** method returns all the element of specified name.

The syntax of the getElementsByName() method is given below:

1. document.getElementsByName("name")

Here, name is required.

Example of document.getElementsByName() method

In this example, we going to count total number of genders. Here, we are using getElementsByName() method to get all the genders.

```
<script type="text/javascript">
function totalelements()
{
var allgenders=document.getElementsByName("gender");
alert("Total Genders:"+allgenders.length);
}
</script>
<form>
Male:<input type="radio" name="gender" value="male">
Female:<input type="radio" name="gender" value="female">
<input type="button" onclick="totalelements()" value="Total Genders">
</form>
```

Output of the above example

Male: ☐ Female: ☐ Total Genders

document.getElementsByTagName() method

The **document.getElementsByTagName()** method returns all the element of specified tag name.

The syntax of the getElementsByTagName() method is given below:

1. document.getElementsByTagName("name")

Here, name is required.

Example of document.getElementsByTagName() method

In this example, we going to count total number of paragraphs used in the document. To do this, we have called the document.getElementsByTagName("p") method that returns the total paragraphs.

```
<script type="text/javascript">
function countpara(){
var totalpara=document.getElementsByTagName("p");
alert("total p tags are: "+totalpara.length);
}
</script>
<p>This is a pragraph</p>
<p>Here we are going to count total number of paragraphs by getElementByTagName() method.</p>
<p>Let's see the simple example</p>
<button onclick="countpara()">count paragraph</button>
```

Output of the above example

This is a paragraph

Here we are going to count total number of paragraphs by getElementByTagName() method.

Let's see the simple example

count paragraph

The History Object:

History Object

The history object contains the URLs visited by the user (within a browser window).

The history object is part of the window object and is accessed through the window.history property.

Note: There is no public standard that applies to the history object, but all major browsers support it.

History Object Properties

Property	Description
length	Returns the number of URLs in the history list

History length Property

Example

Get the number of URLs in the history list:

```
<!DOCTYPE html>
<html>
<body>
<p>Number of URLs in history list:</p>
<p id="demo"></p>
<p><strong>Note:</strong> This example is opened in a new frame and will be treated as a new
"session".</p>
<script>
var x = history.length;
document.getElementById("demo").innerHTML = x;
</script></body></html>
o/p: Number of URLs in history list:
1
```

Note: This example is opened in a new frame and will be treated as a new "session".

History Object Methods

Method	Description
back()	Loads the previous URL in the history list
forward()	Loads the next URL in the history list
go()	Loads a specific URL from the history list

History back() Method

Example

Create a back button on a page:

```
<!DOCTYPE html>
```

```
<html><body>
```

```
<button onclick="goBack()">Go Back</button>
```

<p>Notice that clicking on the Back button here will not result in any action, because there is no previous URL in the history list.</p>

```
<script>
```

```
function goBack() {
```

```
    window.history.back();
```

```
}</script>
```

```
</body></html>
```

o/p: GO back

Notice that clicking on the Back button here will not result in any action, because there is no previous URL in the history list.

History forward() Method

Example

```
<!DOCTYPE html>

<html>

<body>

<button onclick="goForward()">Go Forward</button>

<p>Notice that clicking on the Forward button here will not result in any action, because there is no next URL in the history list.</p>

<script>

function goForward() {
    window.history.forward();
}

</script>

</body></html>
```

o/p: Go forward

Notice that clicking on the Forward button here will not result in any action, because there is no next URL in the history list.

History go() Method

Example

Click on the button to go back two pages:

```
<!DOCTYPE html>

<html><body>

<button onclick="goBack()">Go 2 pages back</button>

<p>Notice that clicking on the "Go 2 pages back" button here will not result in any action, because there is no previous URL in the history list.</p>

<script>

function goBack() {
```

```
    window.history.go(-2);  
}
```

```
</script>
```

```
</body></html>
```

o/p: GO 2 pages back

Notice that clicking on the "Go 2 pages back" button here will not result in any action, because there is no previous URL in the history list.

The Navigator Object:

Navigator Object

The navigator object contains information about the browser.

Note: There is no public standard that applies to the navigator object, but all major browsers support it.

Navigator Object Properties

Property	Description
appName	Returns the code name of the browser
appVersion	Returns the version information of the browser
cookieEnabled	Determines whether cookies are enabled in the browser
geolocation	Returns a Geolocation object that can be used to locate the user's position

language	Returns the language of the browser
onLine	Determines whether the browser is online
platform	Returns for which platform the browser is compiled
product	Returns the engine name of the browser
userAgent	Returns the user-agent header sent by the browser to the server

Navigator Object Methods

Method	Description
javaEnabled()	Specifies whether or not the browser has Java enabled
taintEnabled()	Removed in JavaScript version 1.2. Specifies whether the browser has data tainting enabled

Navigator appCodeName Property:

Example

Get the code name of your browser:

```
<!DOCTYPE html>
```

```
<html>

<body>

<p>Click the button to display the code name of your browser.</p>

<button onclick="myFunction()">Try it</button>

<p id="demo"></p>

<script>

function myFunction() {

    var x = "Browser CodeName: " + navigator.appCodeName;

    document.getElementById("demo").innerHTML = x;

}

</script></body></html>
```

o/p: Click the button to display the code name of your browser.

Try it

Browser CodeName: Mozilla

Navigator appName Property

Example

Get the name of your browser:

```
<!DOCTYPE html>

<html>

<body>


<p>Click the button to display the name of your browser.</p>

<button onclick="myFunction()">Try it</button>

<p id="demo"></p>

<script>

function myFunction() {
```

```
var x = "Browser Name: " + navigator.appName;

document.getElementById("demo").innerHTML = x;

}

</script></body></html>
```

o/p: Click the button to display the name of your browser.

Try it

Browser Name: Netscape

Navigator appVersion Property

Example

Get the version of your browser:

```
<!DOCTYPE html>

<html><body>

<p>Click the button to display the version of your browser.</p>

<button onclick="myFunction()">Try it</button>

<p id="demo"></p>

<script>

function myFunction() {

    var x = "Version info: " + navigator.appVersion;

    document.getElementById("demo").innerHTML = x;

}

</script></body></html>
```

o/p: Click the button to display the version of your browser.

Try it

Version info: 5.0 (Windows NT 10.0) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/64.0.3282.186 Safari/537.36

The Location Object

Location Object

The location object contains information about the current URL.

The location object is part of the window object and is accessed through the window.location property.

Note: There is no public standard that applies to the location object, but all major browsers support it.

Location Object Properties

Property	Description
hash	Sets or returns the anchor part (#) of a URL
host	Sets or returns the hostname and port number of a URL
hostname	Sets or returns the hostname of a URL
href	Sets or returns the entire URL
origin	Returns the protocol, hostname and port number of a URL
pathname	Sets or returns the path name of a URL

[port](#)

Sets or returns the port number of a URL

[protocol](#)

Sets or returns the protocol of a URL

[search](#)

Sets or returns the querystring part of a URL

Location Object Methods

Method

Description

[assign\(\)](#)

Loads a new document

[reload\(\)](#)

Reloads the current document

[replace\(\)](#)

Replaces the current document with a new one

Location hash Property

Example

Return the anchor part of a URL. Assume that the current URL is `http://www.example.com/test.htm#part2`:

```
var x = location.hash;
```

The result of `x` will be:

`#part2`

More "Try it Yourself" examples below.

Definition and Usage

The hash property sets or returns the anchor part of a URL, including the hash sign (#).

Note: When this property is used to set the anchor part, do not include the hash sign (#).

Syntax

Return the hash property:

```
location.hash
```

Set the hash property:

```
location.hash = anchormame
```

Property Values

Value	Type	Description
<i>anchormame</i>	String	Specifies the anchor part of a URL

Technical Details

Return Value:	A String, representing the anchor part of the URL, including the hash sign (#)
----------------------	--

More Examples

Example

Set the anchor part

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>

<p>Click the button to set the anchor part of the current URL to #part5</p>

<button onclick="myFunction()">Try it</button>

<p id="demo"></p>

<script>

function myFunction() {

    location.hash = "part5";

    var x = "The anchor part is now: " + location.hash;

    document.getElementById("demo").innerHTML = x;

}

</script></body></html>
```

o/p: Click the button to set the anchor part of the current URL to #part5

Try it

The anchor part is now: #part5

The Window Object

Window Object

The window object represents an open window in a browser.

If a document contain frames (<iframe> tags), the browser creates one window object for the HTML document, and one additional window object for each frame.

Note: There is no public standard that applies to the Window object, but all major browsers support it.

Window Object Properties

Property	Description
----------	-------------

<u>closed</u>	Returns a Boolean value indicating whether a window has been closed or not
<u>console</u>	Returns a reference to the Console object, which provides methods for logging information to the browser's console (<u>See Console object</u>)
<u>defaultStatus</u>	Sets or returns the default text in the statusbar of a window
<u>document</u>	Returns the Document object for the window (<u>See Document object</u>)
<u>frameElement</u>	Returns the <iframe> element in which the current window is inserted
<u>frames</u>	Returns all <iframe> elements in the current window
<u>history</u>	Returns the History object for the window (<u>See History object</u>)
<u>innerHeight</u>	Returns the inner height of a window's content area
<u>innerWidth</u>	Returns the inner width of a window's content area
<u>length</u>	Returns the number of <iframe> elements in the current window
<u>localStorage</u>	Allows to save key/value pairs in a web browser. Stores the data with no expiration date

<u>location</u>	Returns the Location object for the window (<u>See Location object</u>)
<u>name</u>	Sets or returns the name of a window
<u>navigator</u>	Returns the Navigator object for the window (<u>See Navigator object</u>)
<u>opener</u>	Returns a reference to the window that created the window
<u>outerHeight</u>	Returns the outer height of a window, including toolbars/scrollbars
<u>outerWidth</u>	Returns the outer width of a window, including toolbars/scrollbars
<u>pageXOffset</u>	Returns the pixels the current document has been scrolled (horizontally) from the upper left corner of the window
<u>pageYOffset</u>	Returns the pixels the current document has been scrolled (vertically) from the upper left corner of the window
<u>parent</u>	Returns the parent window of the current window
<u>screen</u>	Returns the Screen object for the window (<u>See Screen object</u>)

[screenLeft](#)

Returns the horizontal coordinate of the window relative to the screen

[screenTop](#)

Returns the vertical coordinate of the window relative to the screen

[screenX](#)

Returns the horizontal coordinate of the window relative to the screen

[screenY](#)

Returns the vertical coordinate of the window relative to the screen

[sessionStorage](#)

Allows to save key/value pairs in a web browser. Stores the data for one session

scrollX

An alias of [pageXOffset](#)

scrollY

An alias of [pageYOffset](#)

[self](#)

Returns the current window

[status](#)

Sets or returns the text in the statusbar of a window

[top](#)

Returns the topmost browser window

Window Object Methods

Method	Description
<u>alert()</u>	Displays an alert box with a message and an OK button
<u>atob()</u>	Decodes a base-64 encoded string
<u>blur()</u>	Removes focus from the current window
<u>btoa()</u>	Encodes a string in base-64
<u>clearInterval()</u>	Clears a timer set with setInterval()
<u>clearTimeout()</u>	Clears a timer set with setTimeout()
<u>close()</u>	Closes the current window
<u>confirm()</u>	Displays a dialog box with a message and an OK and a Cancel button
<u>focus()</u>	Sets focus to the current window

<u>getComputedStyle()</u>	Gets the current computed CSS styles applied to an element
<u>getSelection()</u>	Returns a Selection object representing the range of text selected by the user
<u>matchMedia()</u>	Returns a MediaQueryList object representing the specified CSS media query string
<u>moveBy()</u>	Moves a window relative to its current position
<u>moveTo()</u>	Moves a window to the specified position
<u>open()</u>	Opens a new browser window
<u>print()</u>	Prints the content of the current window
<u>prompt()</u>	Displays a dialog box that prompts the visitor for input
<u>resizeBy()</u>	Resizes the window by the specified pixels
<u>resizeTo()</u>	Resizes the window to the specified width and height
<u>scroll()</u>	Deprecated. This method has been replaced by the <u>scrollTo()</u> method.

<u>scrollBy()</u>	Scrolls the document by the specified number of pixels
<u>scrollTo()</u>	Scrolls the document to the specified coordinates
<u>setInterval()</u>	Calls a function or evaluates an expression at specified intervals (in milliseconds)
<u>setTimeout()</u>	Calls a function or evaluates an expression after a specified number of milliseconds
<u>stop()</u>	Stops the window from loading

Example of alert() in javascript

It displays alert dialog box. It has message and ok button.

```
<script type="text/javascript">
function msg(){
  alert("Hello Alert Box");
}
</script>
<input type="button" value="click" onclick="msg()"/>
```

Output of the above example

Click

Example of confirm() in javascript

It displays the confirm dialog box. It has message with ok and cancel buttons.

```
<script type="text/javascript">
function msg(){
  var v= confirm("Are u sure?");
}
```

```
if(v==true){  
    alert("ok");  
}  
else{  
    alert("cancel");  
} }  
</script>  
<input type="button" value="delete record" onclick="msg()"/>
```

Output of the above example

Click

Example of prompt() in javascript

It displays prompt dialog box for input. It has message and textfield.

```
<script type="text/javascript">  
function msg(){  
    var v= prompt("Who are you?");  
    alert("I am "+v);  
}  
</script>  
  
<input type="button" value="click" onclick="msg()"/>
```

Output of the above example

Click

Example of open() in javascript

It displays the content in a new window.

```
<script type="text/javascript">  
function msg(){  
    open("http://www.javatpoint.com");  
}  
</script>  
<input type="button" value="javatpoint" onclick="msg()"/>
```

Output of the above example

click

Example of setTimeout() in javascript

It performs its task after the given milliseconds.

```
<script type="text/javascript">
function msg(){
setTimeout(
function(){
alert("Welcome to Javatpoint after 2 seconds")
},2000);
}
</script>
<input type="button" value="click" onclick="msg()"/>
```

Output of the above example

Click

user defined function in JavaScript:

An user-defined function saves us from rewriting the same code again and again and helps us to make our application smaller. The given examples will help you to understand how can you put them to work.

JavaScript has so many built-in functions, besides that you can make your own as per the need.

General structure of the user defined function is:

```
function function_name(par1, par2,...){ statement }
```

par1, par2 are the name of the parameter, one function could have one, more than one or no parameter. Parameters could be of any datatype.

Example 1(Simple Function):

```
<html>
<head>
<title>Write your title here </title>
<script type="text/javascript" >
function display()
{
document.write("Hello there");
}
display();
</script>
</head>
</html>
```

Output:

Hello there

Example 2 (Return a Value):

```
<html>
<head>
<title>Write your title here </title>
<script type="text/javascript" >
function add(x,y)
```

```

{
    return (x+y);
}
</script>
</head>
<body>
<script type="text/javascript">
var x=add(12,12);
document.write("Addition is " + x);
</script>
</body>
</html>

```

Output:

Addition is 24

Example 3 (Return a String):

```

<html>
<head>
<title>Write your title here </title>
<script type="text/javascript" >
function display()
{
return("Return Statement");
}
</script>
</head>
<body>
<script type="text/javascript">
document.write(display());
</script>
</body>
</html>

```

Output:

Return Statement

Validation in JavaScript:

JavaScript Form Validation

1. [JavaScript form validation](#)
2. [Example of JavaScript validation](#)
3. [JavaScript email validation](#)

It is important to validate the form submitted by the user because it can have inappropriate values. So validation is must.

The JavaScript provides you the facility to validate the form on the client side so processing will be fast than server-side validation. So, most of the web developers prefer JavaScript form validation.

Through JavaScript, we can validate name, password, email, date, mobile number etc fields.

JavaScript form validation example

In this example, we are going to validate the name and password. The name can't be empty and password can't be less than 6 characters long.

Here, we are validating the form on form submit. The user will not be forwarded to the next page until given values are correct.

```
<script>
function validateform(){
var name=document.myform.name.value;
var password=document.myform.password.value;

if (name==null || name==""){
  alert("Name can't be blank");
  return false;
}else if(password.length<6){
  alert("Password must be at least 6 characters long.");
  return false;
}
}
</script>
<body>
<form name="myform" method="post" action="abc.jsp" onsubmit="return validateform()" >
Name: <input type="text" name="name"><br/>
Password: <input type="password" name="password"><br/>
<input type="submit" value="register">
</form>
```

[Test it Now](#)

JavaScript Retype Password Validation

```
<script type="text/javascript">
function matchpass(){
var firstpassword=document.f1.password.value;
var secondpassword=document.f1.password2.value;

if(firstpassword==secondpassword){
return true;
}
else{
alert("password must be same!");
return false;
}
}
</script>

<form name="f1" action="register.jsp" onsubmit="return matchpass()">
Password:<input type="password" name="password" /><br/>
Re-enter Password:<input type="password" name="password2"/><br/>
<input type="submit">
</form>
```

[Test it Now](#)

JavaScript Number Validation

Let's validate the textfield for numeric value only. Here, we are using isNaN() function.

```
<script>
```

```

function validate(){
var num=document.myform.num.value;
if (isNaN(num)){
    document.getElementById("numloc").innerHTML="Enter Numeric value only";
    return false;
}else{
    return true;
}
}
</script>
<form name="myform" onsubmit="return validate()" >
Number: <input type="text" name="num"><span id="numloc"></span><br/>
<input type="submit" value="submit">
</form>

```

[Test it Now](#)

JavaScript validation with image

Let's see an interactive JavaScript form validation example that displays correct and incorrect image if input is correct or incorrect.

```

<script>
function validate(){
var name=document.f1.name.value;
var password=document.f1.password.value;
var status=false;

if(name.length<1){
document.getElementById("nameloc").innerHTML=
"<img src='unchecked.gif'/> Please enter your name";
status=false;
}else{
document.getElementById("nameloc").innerHTML="<img src='checked.gif'/>";
status=true;
}
if(password.length<6){
document.getElementById("passwordloc").innerHTML=
"<img src='unchecked.gif'/> Password must be at least 6 char long";
status=false;
}else{
document.getElementById("passwordloc").innerHTML="<img src='checked.gif'/>";
status=true;
}
return status;
}
</script>

<form name="f1" action="#" onsubmit="return validate()">
<table>
<tr><td>Enter Name:</td><td><input type="text" name="name"/>
<span id="nameloc"></span></td></tr>
<tr><td>Enter Password:</td><td><input type="text" name="password"/>

```

```

        <span id="passwordloc"></span></td></tr>
        <tr><td colspan="2"><input type="submit" value="register"/></td></tr>
    </table>
</form>

```

[Test it Now](#)

Output:

Enter Name:

Enter Password:

JavaScript email validation

We can validate the email by the help of JavaScript.

There are many criteria that need to be follow to validate the email id such as:

- email id must contain the @ and . character
- There must be at least one character before and after the @.
- There must be at least two characters after . (dot).

Let's see the simple example to validate the email field.

```

<script>
function validateemail()
{
    var x=document.myform.email.value;
    var atposition=x.indexOf("@");
    var dotposition=x.lastIndexOf(".");
    if (atposition<1 || dotposition<atposition+2 || dotposition+2>=x.length){
        alert("Please enter a valid e-
        mail address \n atpostion:"+atposition+"\n dotposition:"+dotposition);
        return false;
    }
}
</script>
<body>
<form name="myform" method="post" action="#" onsubmit="return validateemail();">
Email: <input type="text" name="email"><br/>

<input type="submit" value="register">
</form>

```

event & event handling in JavaScript:

What is an Event ?

JavaScript's interaction with HTML is handled through events that occur when the user or the browser manipulates a page.

When the page loads, it is called an event. When the user clicks a button, that click too is an event. Other examples include events like pressing any key, closing a window, resizing a window, etc.

Developers can use these events to execute JavaScript coded responses, which cause buttons to close windows, messages to be displayed to users, data to be validated, and virtually any other type of response imaginable.

Events are a part of the Document Object Model (DOM) Level 3 and every HTML element contains a set of events which can trigger JavaScript Code.

Please go through this small tutorial for a better understanding [HTML Event Reference](#). Here we will see a few examples to understand a relation between Event and JavaScript

HTML/DOM events for JavaScript

HTML or DOM events are widely used in JavaScript code. JavaScript code is executed with HTML/DOM events. So before learning JavaScript, let's have some idea about events.

Events	Description
onclick	occurs when element is clicked.
ondblclick	occurs when element is double-clicked.
onfocus	occurs when an element gets focus such as button, input, textarea etc.
onblur	occurs when form loses the focus from an element.
onsubmit	occurs when form is submitted.
onmouseover	occurs when mouse is moved over an element.
onmouseout	occurs when mouse is moved out from an element (after moved over).
onmousedown	occurs when mouse button is pressed over an element.
onmouseup	occurs when mouse is released from an element (after mouse is pressed).
onload	occurs when document, object or frameset is loaded.
onunload	occurs when body or frameset is unloaded.
onscroll	occurs when document is scrolled.
onresize	occurs when document is resized.
onreset	occurs when form is reset.
onkeydown	occurs when key is being pressed.
onkeypress	occurs when user presses the key.
onkeyup	occurs when key is released.

onclick Event Type

This is the most frequently used event type which occurs when a user clicks the left button of his mouse. You can put your validation, warning etc., against this event type.

Example

Try the following example.

```
<html>

<head>

  <script type="text/javascript">

    <!--

    function sayHello() {

      alert("Hello World")

    }
```

```
//-->

</script>

</head>

<body>

<p>Click the following button and see result</p>

  <form>

    <input type="button" onclick="sayHello()" value="Say Hello" />

  </form>

</body>

</html>
```

Output

Click the following button and see result

SayHello

onsubmit Event type

onsubmit is an event that occurs when you try to submit a form. You can put your form validation against this event type.

Example

The following example shows how to use onsubmit. Here we are calling a **validate()** function before submitting a form data to the webserver. If **validate()** function returns true, the form will be submitted, otherwise it will not submit the data.

Try the following example.

```
<html>

<head>

  <script type="text/javascript">

    <!--

    function validation() {

      all validation goes here
```

```

.....

    return either true or false

}

//-->

</script>

</head>

<body>

    <form method="POST" action="t.cgi" onsubmit="return validate()">

        .....

        <input type="submit" value="Submit" />

    </form>

</body>

</html>

```

onmouseover and onmouseout

These two event types will help you create nice effects with images or even with text as well. The **onmouseover** event triggers when you bring your mouse over any element and the **onmouseout** triggers when you move your mouse out from that element. Try the following example.

```

<html>

<head>

    <script type="text/javascript">

        <!--

        function over() {

            document.write ("Mouse Over");

        }

        function out() {

            document.write ("Mouse Out");

        }
    </script>

```

```
    //-->

</script>

</head>

<body>

  <p>Bring your mouse inside the division to see the result:</p>

  <div onmouseover="over()" onmouseout="out()">

    <h2> This is inside the division </h2>

  </div>

</body>

</html>
```

Output

Bring your mouse inside the division to see the result:
This is inside the division